

Experiment -7

AIM: Write a c program to implement FCFS algorithm of CPU Scheduling.

```
#include <stdio.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

void sort_by_arrival(struct Process p[], int n) {
    struct Process temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].arrival_time > p[j + 1].arrival_time) {
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n, i;
    struct Process p[20];
    float total_waiting_time = 0, total_turnaround_time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time for P%d: ", i + 1);
        scanf("%d", &p[i].arrival_time);
        printf("Enter burst time for P%d: ", i + 1);
        scanf("%d", &p[i].burst_time);
    }
    sort_by_arrival(p, n);
    int current_time = 0;
    for (i = 0; i < n; i++) {
        if (current_time < p[i].arrival_time) {
```

```

        current_time = p[i].arrival_time; // idle until process arrives
    }
    p[i].waiting_time = current_time - p[i].arrival_time;
    p[i].completion_time = current_time + p[i].burst_time;
    p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
    current_time = p[i].completion_time;

    total_waiting_time += p[i].waiting_time;
    total_turnaround_time += p[i].turnaround_time;
}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
        p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}
printf("\nGantt Chart:\n ");
for (i = 0; i < n; i++) {
    printf("-----");
}
printf("-\n|");
for (i = 0; i < n; i++) {
    printf(" P%d |", p[i].pid);
}
printf("\n ");
for (i = 0; i < n; i++) {
    printf("-----");
}
printf("-\n");
printf("%d", p[0].arrival_time > 0 ? p[0].arrival_time : 0);
for (i = 0; i < n; i++) {
    printf("    %d", p[i].completion_time);
}
printf("\n");
printf("\nAverage Waiting Time: %.2f", total_waiting_time / n);
printf("\nAverage Turnaround Time: %.2f\n", total_turnaround_time / n);

return 0;
}

```

OUTPUT:

```
Enter number of processes: 5
Enter arrival time for P1: 2
Enter burst time for P1: 12
Enter arrival time for P2: 0
Enter burst time for P2: 24
Enter arrival time for P3: 3
Enter burst time for P3: 6
Enter arrival time for P4: 4
Enter burst time for P4: 9
Enter arrival time for P5: 1
Enter burst time for P5: 4
```

Process	AT	BT	CT	TAT	WT
P2	0	24	24	24	0
P5	1	4	28	27	23
P1	2	12	40	38	26
P3	3	6	46	43	37
P4	4	9	55	51	42

Gantt Chart:

```
-----
| P2 | | P5 | | P1 | | P3 | | P4 | |
-----
0      24      28      40      46      55
```

Average Waiting Time: 25.60

Average Turnaround Time: 36.60

Experiment -8

AIM: Write a c program to implement Round Robin Algorithm.

```
#include <stdio.h>

#include <stdbool.h>

struct Process {

    int pid;

    int arrival_time;

    int burst_time;

    int remaining_time;

    int completion_time;

    int waiting_time;

    int turnaround_time;

    bool is_completed;

};

int main() {

    int n, time_quantum;

    struct Process p[20];

    int i, time = 0, completed = 0;

    int queue[100], front = 0, rear = 0;

    float total_waiting = 0, total_turnaround = 0;

    bool in_queue[20] = {false};

    printf("Enter number of processes: ");

    scanf("%d", &n);

    // Input processes

    for (i = 0; i < n; i++) {

        p[i].pid = i + 1;
```

```
printf("Enter arrival time for P%d: ", p[i].pid);
scanf("%d", &p[i].arrival_time);
printf("Enter burst time for P%d: ", p[i].pid);
scanf("%d", &p[i].burst_time);
p[i].remaining_time = p[i].burst_time;
p[i].is_completed = false;
}

printf("Enter time quantum: ");
scanf("%d", &time_quantum);
for (i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (p[j].arrival_time > p[j + 1].arrival_time) {
            struct Process temp = p[j];
            p[j] = p[j + 1];
            p[j + 1] = temp;
        }
    }
}
queue[rear++] = 0;
in_queue[0] = true;

printf("\nGantt Chart:\n ");
int gantt[200], gantt_len = 0;

while (completed != n) {
    int idx = queue[front++];
    if (p[idx].remaining_time > 0) {
        int exec_time = (p[idx].remaining_time > time_quantum) ? time_quantum :
p[idx].remaining_time;
        time += exec_time;
```

```
p[idx].remaining_time -= exec_time;
gantt[gantt_len++] = p[idx].pid;
for (i = 0; i < n; i++) {
    if (!in_queue[i] && p[i].arrival_time <= time && p[i].remaining_time > 0) {
        queue[rear++] = i;
        in_queue[i] = true;
    }
}
if (p[idx].remaining_time > 0) {
    queue[rear++] = idx;
} else {
    p[idx].completion_time = time;
    p[idx].turnaround_time = p[idx].completion_time - p[idx].arrival_time;
    p[idx].waiting_time = p[idx].turnaround_time - p[idx].burst_time;
    total_turnaround += p[idx].turnaround_time;
    total_waiting += p[idx].waiting_time;
    p[idx].is_completed = true;
    completed++;
}
}
if (front == rear && completed != n) {
    for (i = 0; i < n; i++) {
        if (!p[i].is_completed) {
            queue[rear++] = i;
            in_queue[i] = true;
            time = p[i].arrival_time > time ? p[i].arrival_time : time;
            break;
        }
    }
}
}
```

```

printf(" ");
for (i = 0; i < gantt_len; i++) {
    printf("-----");
}
printf("-\n|");
for (i = 0; i < gantt_len; i++) {
    printf(" P%d  |", gantt[i]);
}
printf("\n ");
for (i = 0; i < gantt_len; i++) {
    printf("-----");
}
printf("-\n");
int marker = 0;
printf("0");
for (i = 0; i < gantt_len; i++) {
    marker += (p[gantt[i] - 1].burst_time < time_quantum) ? p[gantt[i] - 1].burst_time :
time_quantum;
    printf("    %d", marker);
}
printf("\n");
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
        p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}

printf("\nAverage Waiting Time: %.2f", total_waiting / n);
printf("\nAverage Turnaround Time: %.2f\n", total_turnaround / n);

return 0;

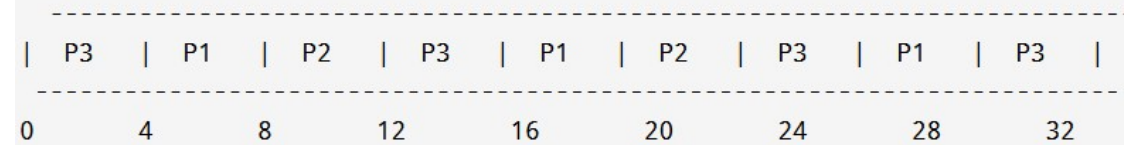
```

}

OUTPUT:

```
Enter number of processes: 3
Enter arrival time for P1: 1
Enter burst time for P1: 10
Enter arrival time for P2: 2
Enter burst time for P2: 8
Enter arrival time for P3: 0
Enter burst time for P3: 13
Enter time quantum: 4
```

Gantt Chart:



Process AT BT CT TAT WT

P3 0 13 31 31 18

P1 1 10 30 29 19

P2 2 8 24 22 14

Average Waiting Time: 17.00

Average Turnaround Time: 27.33

Experiment -9

AIM: Write a menu based c program to implement shortest time remaining first and pre-emptive priority based scheduling algorithm .

```
#include <stdio.h>

#include <stdbool.h>

#define MAX 100

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int priority;
    int completion_time;
    int turnaround_time;
    int waiting_time;
    bool is_completed;
};

void calculate_and_display(struct Process p[], int n) {
    float total_wt = 0, total_tat = 0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for (int i = 0; i < n; i++) {
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
        total_wt += p[i].waiting_time;
        total_tat += p[i].turnaround_time;

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
            p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }
}
```

```
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);

printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

}

void SRTF(struct Process p[], int n) {
    int time = 0, completed = 0;
    int min_idx;
    while (completed < n) {
        int shortest = 1e9;
        min_idx = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].arrival_time <= time && !p[i].is_completed && p[i].remaining_time < shortest &&
                p[i].remaining_time > 0) {
                shortest = p[i].remaining_time;
                min_idx = i;
            }
        }

        if (min_idx == -1) {
            time++;
            continue;
        }

        p[min_idx].remaining_time--;
        time++;

        if (p[min_idx].remaining_time == 0) {
            p[min_idx].completion_time = time;
            p[min_idx].is_completed = true;
            completed++;
        }
    }
}
```

```
    }  
}  
calculate_and_display(p, n);  
}
```

```
void Preemptive_Priority(struct Process p[], int n) {  
    int time = 0, completed = 0;  
    int min_idx;  
  
    while (completed < n) {  
        int highest_priority = 1e9;  
        min_idx = -1;  
  
        for (int i = 0; i < n; i++) {  
            if (p[i].arrival_time <= time && !p[i].is_completed && p[i].priority < highest_priority &&  
p[i].remaining_time > 0) {  
                highest_priority = p[i].priority;  
                min_idx = i;  
            }  
        }  
  
        if (min_idx == -1) {  
            time++;  
            continue;  
        }  
  
        p[min_idx].remaining_time--;  
        time++;  
  
        if (p[min_idx].remaining_time == 0) {  
            p[min_idx].completion_time = time;  
            completed++;  
        }  
    }  
}
```

```
        p[min_idx].is_completed = true;
        completed++;
    }
}

calculate_and_display(p, n);
}

void reset_processes(struct Process p[], int n, struct Process original[]) {
    for (int i = 0; i < n; i++) {
        p[i] = original[i];
        p[i].remaining_time = p[i].burst_time;
        p[i].is_completed = false;
    }
}

int main() {
    struct Process p[MAX], original[MAX];
    int n, choice;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input process data
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nEnter arrival time for P%d: ", i + 1);
        scanf("%d", &p[i].arrival_time);
        printf("Enter burst time for P%d: ", i + 1);
        scanf("%d", &p[i].burst_time);
        printf("Enter priority for P%d (lower = higher priority): ", i + 1);
        scanf("%d", &p[i].priority);
    }
}
```

```
p[i].remaining_time = p[i].burst_time;
p[i].is_completed = false;

original[i] = p[i]; // keep a copy to reset later
}

do {
    printf("\n\n--- CPU Scheduling Menu ---\n");
    printf("1. Shortest Remaining Time First (SRTF)\n");
    printf("2. Preemptive Priority Scheduling\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    reset_processes(p, n, original); // restore original states before each run

    switch (choice) {
        case 1:
            printf("\n--- Executing SRTF Scheduling ---\n");
            SRTF(p, n);
            break;
        case 2:
            printf("\n--- Executing Preemptive Priority Scheduling ---\n");
            Preemptive_Priority(p, n);
            break;
        case 3:
            printf("\nExiting Program.\n");
            break;
        default:
            printf("Invalid choice. Please select again.\n");
    }
}
```

```
} while (choice != 3);
```

```
return 0;
```

```
}
```

OUTPUT:

```
Enter number of processes: 3
```

```
Enter arrival time for P1: 0
```

```
Enter burst time for P1: 2
```

```
Enter priority for P1 (lower = higher priority): 4
```

```
Enter arrival time for P2: 2
```

```
Enter burst time for P2: 4
```

```
Enter priority for P2 (lower = higher priority): 12
```

```
Enter arrival time for P3: 1
```

```
Enter burst time for P3: 5
```

```
Enter priority for P3 (lower = higher priority): 0
```

```
---- CPU Scheduling Menu ----
```

```
1. Shortest Remaining Time First (SRTF)
```

```
2. Preemptive Priority Scheduling
```

```
3. Exit
```

```
Enter your choice: 2
```

```
--- Executing Preemptive Priority Scheduling ---
```

PID	AT	BT	CT	TAT	WT
P1	0	2	7	7	5
P2	2	4	11	9	5
P3	1	5	6	5	0

```
Average Waiting Time: 3.33
```

```
Average Turnaround Time: 7.00
```

```
---- CPU Scheduling Menu ----  
1. Shortest Remaining Time First (SRTF)  
2. Preemptive Priority Scheduling  
3. Exit  
Enter your choice: 1
```

```
--- Executing SRTF Scheduling ---
```

PID	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	2	4	6	4	0
P3	1	5	11	10	5

Average Waiting Time: 1.67

Average Turnaround Time: 5.33

Experiment -10

AIM: Write a program for Banker's algo which provide safe sequence of execution of the processes or print if there is a deadlock.

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 20

int main() {
    int n, m, i, j, k;
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX], avail[MAX];
    int finish[MAX] = {0}, safeSeq[MAX];
    bool found;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - alloc[i][j]; // Calculate Need
        }
    }

    // Input Available Resources
    printf("\nEnter Available Resources: ");
    for (j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }
```



```

// Banker's Algorithm
int count = 0;
while (count < n) {
    found = false;
    for (i = 0; i < n; i++) {
        if (!finish[i]) {
            bool can_allocate = true;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) {
                    can_allocate = false;
                    break;
                }
            }
            if (can_allocate) {
                // Simulate resource allocation
                for (k = 0; k < m; k++) {
                    avail[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = 1;
                found = true;
            }
        }
    }
}

if (!found) {
    printf("\n ❌ Deadlock detected! No safe sequence exists.\n");
    return 0;
}

// Output Safe Sequence
printf("\n ✅ System is in a SAFE STATE.\nSafe Sequence: ");
for (i = 0; i < n; i++) {
    printf("P%d", safeSeq[i]);
    if (i != n - 1)
        printf(" -> ");
}
printf("\n");

return 0;
}

```

OUTPUT 1:

```
Enter number of processes: 4
Enter number of resource types: 3

Enter Allocation Matrix:
P0: 1 2 1
P1: 0 3 1
P2: 1 3 5
P3: 2 0 0

Enter Maximum Matrix:
P0: 3 4 2
P1: 1 3 2
P2: 2 4 5
P3: 3 1 2

Enter Available Resources: 1 2 1

✅ System is in a SAFE STATE.
Safe Sequence: P1 -> P2 -> P3 -> P0
```

OUTPUT 2:

```
Enter number of processes: 4
Enter number of resource types: 3

Enter Allocation Matrix:
P0: 1 2 0
P1: 0 2 1
P2: 0 3 4
P3: 1 0 3

Enter Maximum Matrix:
P0: 4 7 2
P1: 4 6 4
P2: 6 8 4
P3: 3 5 7

Enter Available Resources: 1 2 1

❌ Deadlock detected! No safe sequence exists.
```